Structural Defects Network: MLOps project using ResNet-18 based binary image classifier

Course ID.: CPE393 Machine Learning Operations

Members:

Thanapat Sittiyodying
ID: 64070501067 <than.sitt@mail.kmutt.ac.th>
Kwinyarut Poungsangthanakul
ID: 65070503404 <kwin.poun@mail.kmutt.ac.th>
Panyawut Piyasirinanan
ID: 65070503423 <pany.piya@mail.kmutt.ac.th>
Nanphat Tongsirisukool
ID: 65070503454 <nanp.tong@mail.kmutt.ac.th>
Yuil Tripathee
ID: 65070503480 <yuil.trip@mail.kmutt.ac.th>

Submitted To:

Department of Computer Engineering
in partial fulfillment of the requirements for the course of
CPE393 Machine Learning Operations.

Instructor:

Asst. Prof. Dr. Aye Hninn Khine
Asst. Prof. Dr. Santitham Prom-on

KMUTT
May, 2025

ABSTRACT

In the wake of recent earthquakes, the need for rapid and accurate structural damage assessment has grown more urgent. This project introduces an end-to-end MLOps pipeline for automating the detection of cracks in concrete surfaces using deep learning techniques. Leveraging the publicly available SDNET2018 dataset from Kaggle, which comprises over 56,000 labeled images of cracked and non-cracked concrete, the team developed and compared several classification models, including a custom CNN, ResNet-18, and MobileNetV3.

The dataset was split into training, validation, and test sets using 70-15-15 and 60-40 strategies, while addressing class imbalance through undersampling. Extensive image preprocessing and augmentation enhanced model robustness. Hyperparameter tuning was conducted using Optuna, and Soft F1 Loss was adopted to better handle class imbalance. Among all models, the ResNet-18 with optimized parameters achieved a validation F1-score of 0.7559, while MobileNetV3 reached a test accuracy of 91% and an F1-score of 0.75 under a mildly imbalanced configuration.

The system was deployed using FastAPI and containerized via Docker, with CI/CD support from GitHub Actions. User feedback was incorporated into a continuous learning loop managed by Apache Airflow and MLflow. This integration of AI and DevOps not only streamlines real-world deployment but also enables scalable, adaptive infrastructure monitoring. The project lays a foundation for future enhancements such as drift detection and object-level crack localization.

Keywords: *Crack Detection, ResNet-18, MobileNetV3, MLOps, SDNET2018, Image Classification, FastAPI, Docker, Airflow, MLflow, CI/CD, Concrete Health Monitoring*

Table of Contents

List of Figures

List of Tables

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

As the recent earthquake in Thailand caused significant structural damage, particularly to high-rise buildings which led to widespread concrete cracking and breakage. [1] Therefore, the need for efficient and reliable damage assessment has become more critical than ever. Structural defects such as cracks and spalling not only undermine the integrity of infrastructure but also pose serious risks to public safety if not detected and repaired promptly.

Traditional inspection methods rely heavily on manual labor which are time-consuming and are prone to human error making them impractical for large scale assessments especially in post-disaster scenarios. By that we want to enhance the current method by using modern technology from this project instead. [2]

This project proposes an AI-powered defect detection system that leverages deep learning to automate the identification and localization of structural damage from images. By utilizing pre-trained models such as ResNet [3], EfficientNet[4] and MobileNetV3[5]. The system classifies images as either cracked or non-cracked and applies object detection techniques for precise localization. This approach not only improves the speed and accuracy of structural assessments but also supports engineers and decision-makers in prioritizing repairs, ultimately enhancing infrastructure resilience after natural disasters.

1.2 Project Scope and Goals

The goal of this project is to enhance structural health monitoring through automated crack detection, reducing manual inspection and improving safety. To achieve this, the project will aim to:

1. Develop a classification model (ResNet, EfficientNet and MobileNetV3) in order to distinguish between cracked and non-cracked surfaces with high accuracy.

2. Implement object detection and segmentation using Faster R-CNN for precise localization of cracks.

3. Preprocess and enhance data, including image augmentation and feature extraction for improved model performance.

4. Train and evaluate models using relevant performance metrics such as accuracy, precision, recall, F1-score, AUC-ROC and Confusion Matrix.

5. Deploy the best-performing model via Flask/FastAPI, Docker and cloud platforms (AWS, GCP, Azure or Hugging Face Spaces) for real-world applications.

6. Explore optional CI/CD integration, including automated model retraining, performance monitoring, and deployment pipelines.

This project will contribute to efficient and scalable crack detection, improving infrastructure maintenance and safety through AI-driven automation.

1.3 Data Description

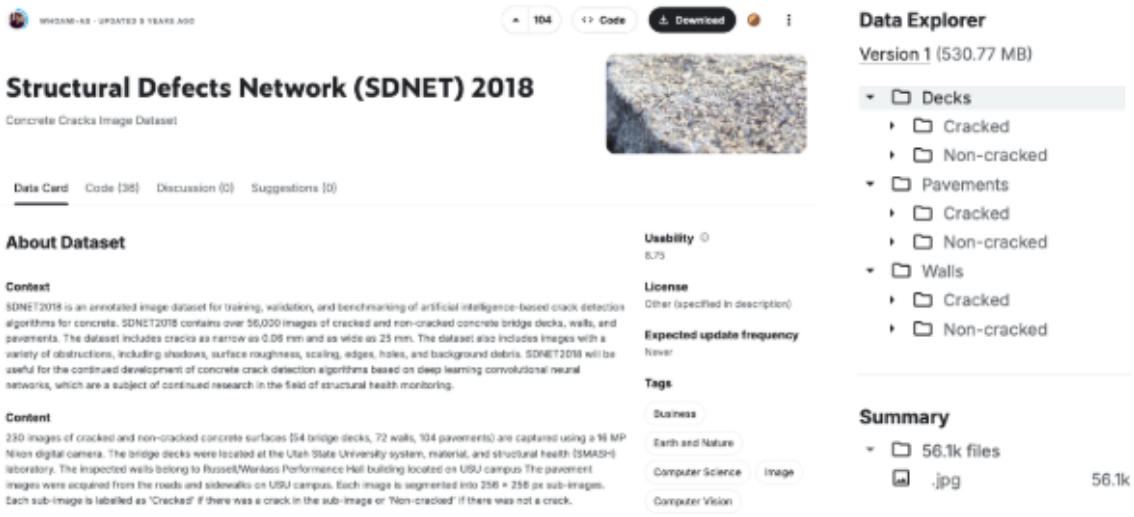

Figure 1.1: Structural Defects Network (SDNET)

In order to implement this project, we use the dataset from Kaggle (URL: https://www.kaggle.com/datasets/aniruddhsharma/structural-defects-network-concrete-crack-images). This dataset is called "Structural Defects Network (SDNET) 2018".

The SDNET2018 dataset is a large, labeled image dataset used to train and test AI models for detecting cracks in concrete. It contains over 56,000 images of both cracked and non-cracked concrete surfaces which include bridge decks, walls and pavements. The

cracks range in width from 0.06 mm to 25 mm and include various real-world conditions like shadows, rough surfaces and debris. Additionally, Each image is segmented into 256×256 px sub-images. Each sub-image is labelled as 'Cracked' if there was a crack in the sub-image or 'Non-cracked' if there was not a crack. Moreover, this dataset supports the development of deep learning models for concrete crack detection and is widely used in structural health monitoring research.

1.4 System Implementation and Source Code

The complete source code, model training pipeline, MLOps integration and the documentation are available on the following link :

https://github.com/hammer-pp/Structural-Defects-Network-MLOps

1.5 Keywords

Deep Learning, Crack Detection, Concrete Surface Inspection, Structural Health Monitoring, Image Classification, Convolutional Neural Networks (CNN), Pre-trained Models (ResNet, MobileNet), Image Augmentation, Feature Extraction, Model Evaluation Metrics (Accuracy, Precision, Recall, F1-score, AUC-ROC and Confusion Matrix), Flask / FastAPI Deployment, Cloud Computing (AWS, GCP, Azure) and CI / CD for Machine Learning

CHAPTER 2

IMPLEMENTATION METHODOLOGY

2.1 Data Pre-processing

2.1.a Handling Class Imbalance

As the original SDNET dataset exhibits significant class imbalance, with non-cracked images comprising approximately 85% of the data. To mitigate this issue and improve model sensitivity to cracks, two resampling strategies were applied:

Balanced 50/50 dataset Random undersampling matched the number of non-cracked images to the number of cracked images. This ensures equal representation and helps prevent model bias.

Balanced 40/60 dataset A less aggressive undersampling approach was used, setting a cracked-to-non-cracked ratio of 40:60. This retains more samples while reducing imbalance.

These resampled datasets allow for performance comparisons under different balance conditions, ensuring the model's robustness across varying class distributions.

2.1.b Image Preprocessing

Before being fed into the model, images underwent several preprocessing steps aimed at enhancing their quality and diversity:

Resizing All images were resized to the same resolution for uniformity.

Denoising A mild blur was applied to reduce noise while preserving important features such as crack edges.

Normalization Pixel values were scaled to a standardized range to help the model learn more effectively.

These steps ensured that the input data was clean, consistent, and suitable for deep learning.

2.2 Data Augmentation

To enhance the robustness of the model and prevent overfitting, data augmentation techniques were applied to the training image, which included:

Random horizontal flipping to simulate mirrored crack patterns.

Small-angle rotation $\pm 15°$ to mimic slight variations in camera angle.

Brightness and contrast adjustments to reflect differing lighting environments.

These augmentations simulate variations in lighting and camera angles, helping the model generalize better to new, unseen data.

2.3 Dataset Construction and Splitting

The dataset used in this project was derived from SDNET, which consists of images categorized into three surface types: decks, pavements, and walls. All images were compiled into a single dataset and standardized in size to 256×256 pixels to ensure consistency and compatibility with the model input requirements. To facilitate model evaluation, the dataset was divided into three subsets:

- 70% for training
- 15% for testing
- 15% for validation

A fixed random seed was applied during splitting to ensure the results are reproducible.

2.4 EDA (Exploratory Data Analysis) findings

To gain a deeper understanding of the dataset before model training, several exploratory analyses were conducted. These analyses focused on class distributions, dataset balancing, and visual inspection of the data.

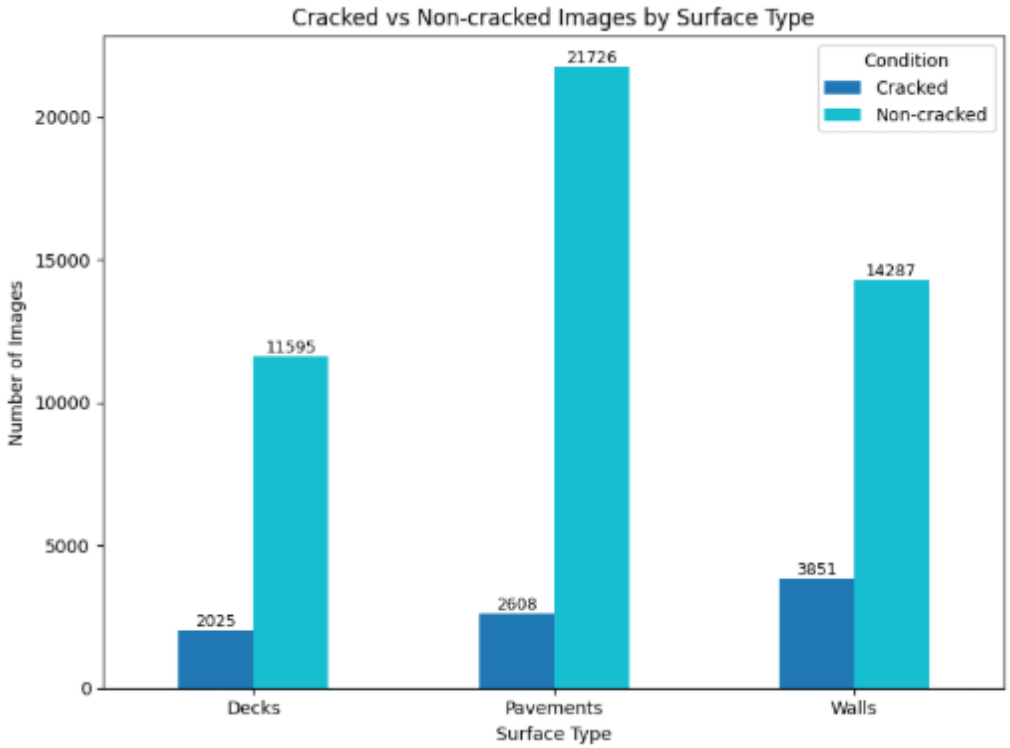

Figure 2.1: Cracked vs Non-Cracked Images by Surface Type

2.5 Class Distribution

2.5.a Class Distribution by Surface Type

A bar chart was plotted to visualize the number of cracked and non-cracked images across the three surface types: decks, pavements, and walls. The analysis revealed a noticeable class imbalance in all surface categories, with non-cracked images significantly outnumbering cracked ones. This imbalance was most pronounced in pavement images. Understanding this distribution was essential for planning data balancing strategies and selecting appropriate evaluation metrics.

2.5.b Class Distribution Across Datasets

Another bar chart compares the class distributions in the original dataset and the two resampled training datasets: 50/50 balanced and 40/60 balanced.

- In the original dataset, non-cracked images constituted approximately 90% of the data.
- The 50/50 dataset ensured perfect class balance for fair model training.
- The 40/60 dataset retained a slight imbalance to reflect realistic field data better while mitigating extreme class skew.

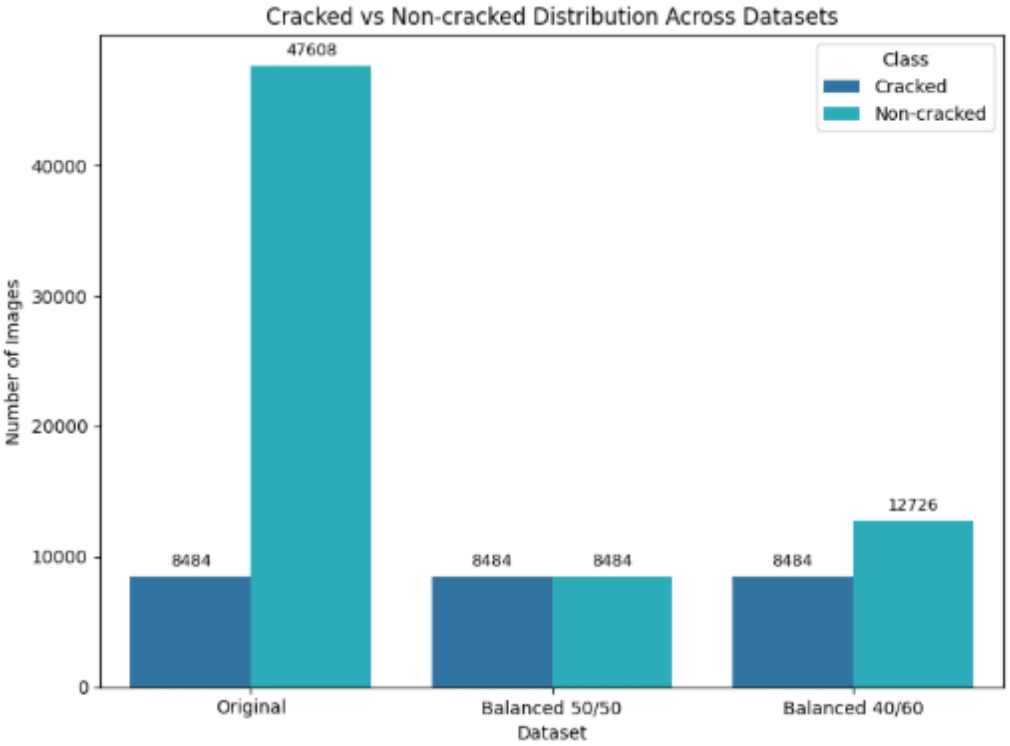

Figure 2.2: Cracked vs Non-Cracked Images Across Datasets

2.6 Sample Images Overview

Representative images were displayed to give a qualitative sense of the dataset. Samples from both cracked and non-cracked categories were shown across different surface types. These visual examples highlight the diversity in texture, lighting, and crack appearance, and they emphasize the importance of robust preprocessing and augmentation techniques.

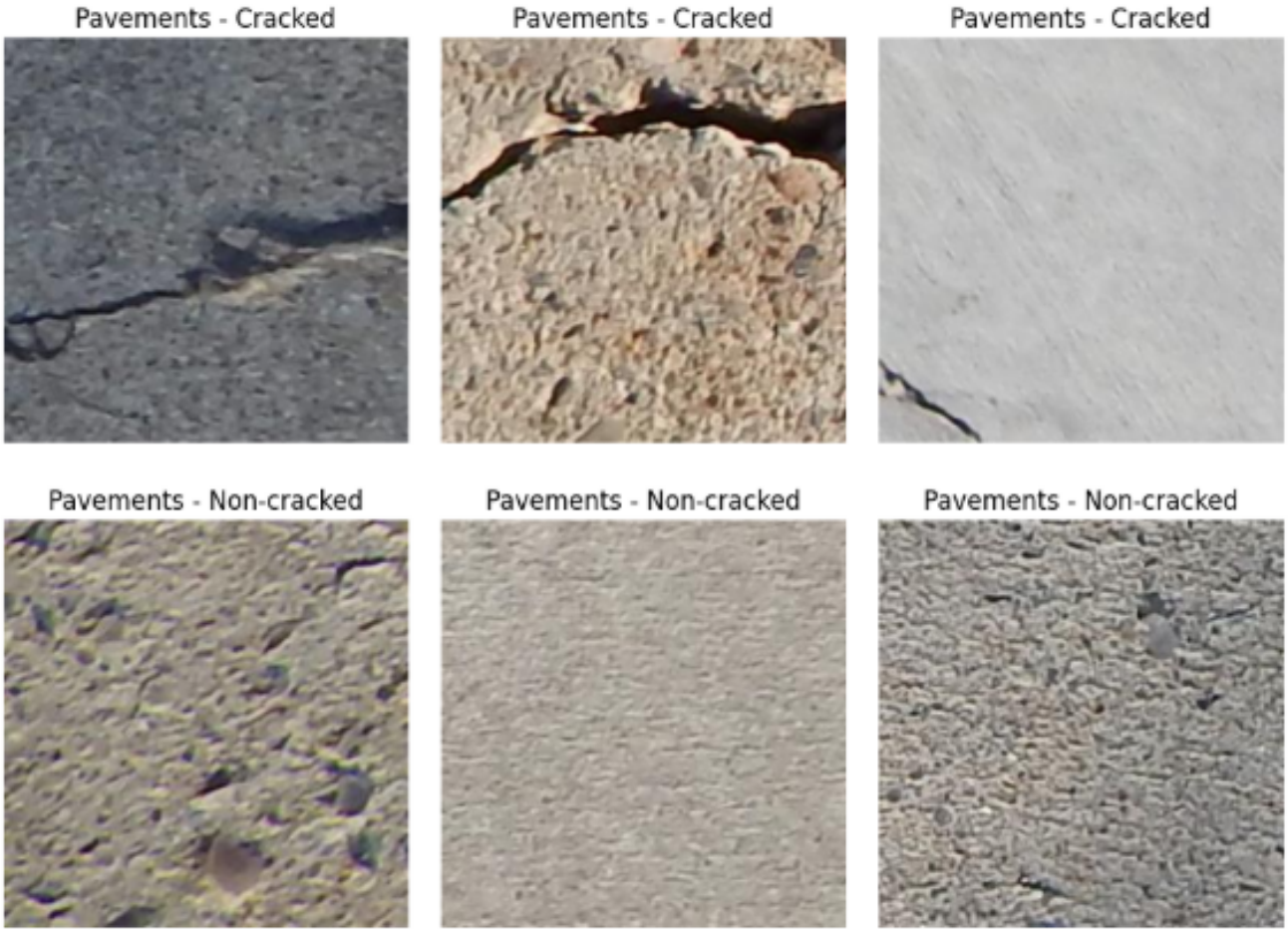

Figure 2.3: Datasets sample overview

CHAPTER 3

MODEL ARCHITECTURE

3.1 Classification Models

3.1.a Custom CNN (Convolutional Neural Network)

A Custom Convolutional Neural Network (CNN) is a deep learning model specifically designed and built from scratch for a particular task, such as image classification. Unlike pre-trained models, a custom CNN is architected and trained entirely on the target dataset. It typically consists of multiple layers, including convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for classification.

Using a custom CNN provides full control over the architecture, allowing it to be optimized specifically for detecting cracks in concrete surfaces. By tailoring kernel sizes, layer depths, and activation functions, the model can be better adapted to the unique characteristics of the dataset, such as varying crack widths, textures, and lighting conditions. This flexibility can lead to improved performance when pre-trained models are not fully aligned with the problem domain.

3.1.b ResNet (Residual Network)

ResNet (Residual Network) is a deep convolutional neural network architecture introduced by Microsoft Research in 2015. It solves the problem of vanishing gradients in very deep networks by using "residual connections" or "skip connections," which allow the network to learn identity mappings and preserve gradient flow during back-propagation. This enables ResNet to be trained effectively even with hundreds of layers, making it highly accurate for image classification tasks.[3]

ResNet is ideal for crack detection because it combines high accuracy with computational efficiency. By leveraging pre-trained weights from large-scale datasets like ImageNet, ResNet can transfer learned features to the crack detection task, accelerating training and improving generalization. Its ability to capture both low-level textures and

high-level patterns makes it well-suited for identifying subtle crack features in concrete surfaces.

3.1.c MobileNetv3

MobileNetV3 is a lightweight convolutional neural network (CNN) developed by Google which is optimized for both speed and accuracy, especially on mobile and edge devices. It is an enhanced version of its predecessors, MobileNetV1 and V2.[6][5]

Moreover, this model also comes in two variants:

1. MobileNetV3-Large for higher accuracy
2. MobileNetV3-Small for faster performance on low-power devices.

The architecture combines depth-wise separable convolutions in order to reduce computation, squeeze-and-excitation (SE) blocks to emphasize important features and a new activation function called hard-swish which improves speed with minimal accuracy loss. These features make MobileNetV3 highly efficient for image classification tasks.

Therefore, we decided to implement this model. As it is particularly suitable for the SDNET2018 dataset which contains over 56,000 images of cracked and non-cracked concrete surfaces including various challenging real-world conditions such as shadows, dirt and surface roughness. MobileNetV3,Äôs ability to extract meaningful features while maintaining a lightweight structure allows it to perform well even under such noisy conditions.

Furthermore, its efficiency makes it ideal for real-time deployment in post-disaster scenarios where fast and reliable crack detection is critical. As we want to deploy on our website. MobileNetV3 enables accurate and fast structural assessments which help engineers and authorities prioritize repairs and enhance infrastructure resilience.

Table 3.1 compares all 3 classification models in a summary.

3.2 Object Detection Models

3.2.a Faster R-CNN

Faster R-CNN (Region-based Convolutional Neural Network) is a powerful object detection model designed to both identify and localize objects within an image. Unlike simple classification models that only predict whether an image contains a crack or not, Faster R-CNN can detect the exact location of structural defects by drawing bounding boxes around them. The model works in two main stages.[7]

First, it uses a Region Proposal Network (RPN) to quickly scan the image and suggest possible areas where cracks might be found. Then, it uses a classification and regression

Model	Architecture Highlights	Key Strengths	Why It's Different / Suitable for Task
Custom CNN	Manually designed CNN with convolution, pooling, and dense layers tuned to task	Fully adaptable; focused on specific features; controlled complexity	Tailored to crack characteristics; outperforms when pre-trained models don't fit perfectly
ResNet	Deep CNN with residual (skip) connections that allow identity mapping and deep training	High accuracy; pre-trained on large datasets; captures low- and high-level features	Deep residual learning enables better gradient flow and generalization, ideal for classification
MobileNetV3	Lightweight CNN with depth-wise separable convolutions, SE blocks, and hard-swish	Efficient, fast inference; strong on mobile/edge; robust to noise and real-world variability	Optimized for real-time, low-resource environments; perfect for noisy datasets and deployment

Table 3.1: Comparison of Model Architectures for Crack Detection in Concrete

network to refine these regions and classify them. This two-stage approach provides high accuracy in detecting small or narrow cracks, even in complex backgrounds like those found in the SDNET2018 dataset.

Faster R-CNN is especially suitable for crack detection in concrete because it can handle images with obstructions, shadows, and rough textures, which are common in the dataset. This makes it ideal for post-earthquake structural inspections, where engineers need to know exactly where the damage is in order to prioritize repairs. By applying Faster R-CNN, the project can move beyond basic classification to a more practical and actionable solution for real-world infrastructure monitoring.

3.3 Model training and Implementation

To develop a robust crack detection system, we employed deep convolutional neural networks using the PyTorch framework. The implementation leverages both pre-trained architectures ResNet and MobileNetV3 to enable transfer learning and improve classification accuracy with limited training data.

3.3.a Libraries and Tools Used

PyTorch (`torch`, `torchvision`) Core framework for defining models, loss functions, optimizers, and performing GPU-accelerated training.

`torchvision.models` Provides access to pre-trained models like resnet50 and mobilenet_v3_large, which are fine-tuned on our dataset.

`torchvision.transforms` Used to preprocess images to match the input format expected by pre-trained models (e.g., resizing to 224×224, normalization using ImageNet statistics).

`pandas` Handles CSV-based dataset annotations.

`PIL` Loads image files for transformation.

`scikit-learn` Calculates performance metrics (accuracy, precision, recall, F1-score, etc.)

`tqdm` Displays progress bars for training and evaluation loops.

3.4 Dataset Preparation and Preprocessing

The dataset, organized into training, validation, and test splits, is managed using a custom CrackDataset class that inherits from `torch.utils.data.Dataset`. This class reads image paths and labels from CSV files, mapping labels ("Cracked" to 1, "Non-cracked" to 0). Images are preprocessed using `torchvision.transforms` to resize them to 224×224 pixels, convert them to tensors, and normalize them with ImageNet statistics (mean: [0.485, 0.456, 0.406], std: [0.229, 0.224, 0.225]). DataLoader objects facilitate batched loading with a batch size of 32, enabling efficient training and evaluation.

```
# Transform to 244 x 244
transform = transforms.Compose([
  transforms.Resize((224, 224)),
  transforms.ToTensor(),
  transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

3.5 Model Implementation

We experimented with two popular CNN architectures:

ResNet-50 (`models.resnet50`)

MobileNetV3-Large (`models.mobilenet_v3_large(weights=MobileNet_V3_Large_Weights.IMAGENET1K_V1)`)

These models are pre-trained on ImageNet and then fine-tuned for binary classification by replacing the final layer with a fully connected layer of size 2 (for 2 classes: cracked and non-cracked).

3.5.a Training Configuration

- Loss Function
- Optimizer
- Scheduler

3.5.b `DataLoader` Setup

Data is loaded in batches using `DataLoader`, with separate loaders for training, validation, and testing. Shuffling is applied to the training loader to ensure better generalization.

```
# Load datasets
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32)
test_loader = DataLoader(test_dataset, batch_size=32)
```

3.5.c Device Setup

Training is performed on GPU if available:

```
# Check CUDA GPU availability
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

3.6 Classification Models

3.6.a Custom CNN

Omitted and redacted from documentation.

3.6.b ResNet

The implementation of the ResNet-18 model for crack detection in structural images involves a systematic approach leveraging transfer learning, advanced optimization techniques, and robust evaluation strategies. This discussion outlines the methodology, focusing on the experimental modifications to the loss function, hyperparameter optimization, and learning rate scheduling, as demonstrated in the provided notebook.

3.6.b.1 Model architecture and transfer learning

ResNet-18, a deep convolutional neural network with residual connections, was selected for its balance of computational efficiency and performance. The model, pre-trained on ImageNet, was adapted for binary classification (cracked vs. non-cracked surfaces) by replacing the final fully connected layer with a new layer outputting two classes. This transfer learning approach leverages pre-trained weights to extract relevant features, reducing training time and mitigating overfitting on a potentially limited dataset. The implementation uses PyTorch's **`torchvision.models.resnet18`** with weights set to **`ResNet18_Weights.IMAGENET1K_V1`**, ensuring compatibility with modern PyTorch conventions.

```
import torch
import torch.nn as nn
from torchvision import models, transforms

model = models.resnet18(pretrained=True)
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 2) # 2 classes: Cracked / Non-cracked
```

3.6.b.2 Soft F1 Loss

To address class imbalance and prioritize both precision and recall, a custom SoftF1Loss was introduced instead of the standard Cross-Entropy Loss. The Soft F1 Loss computes a differentiable F1-score based on predicted probabilities, defined as:

$$SoftF1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN + \epsilon}$$

where TP, FP, and FN are true positives, false positives, and false negatives calculated from softmax probabilities, and $\epsilon \times 10^{-8}$ prevents division by zero. The loss is then defined as 1 - SoftF1, encouraging the model to maximize the F1-score. This approach is particularly effective for imbalanced datasets, as it directly optimizes for the harmonic mean of precision and recall, aligning with the evaluation metric used (F1-score).

```
class SoftF1Loss(nn.Module):
  def __init__(self):
    super().__init__()

  def forward(self, logits, targets):
    probs = torch.softmax(logits, dim=1)[:, 1]
    targets = targets.float()
```

```
    TP = (probs * targets).sum()
    FP = (probs * (1 - targets)).sum()
    FN = ((1 - probs) * targets).sum()

    soft_f1 = 2 * TP / (2 * TP + FP + FN + 1e-8)
    return 1 - soft_f1 # we want to minimize loss, so use 1 - F1
```

3.6.b.3 Optimizer and Hyperparameter Tuning

The Adam optimizer was employed for its adaptive learning rate properties, which accelerate gradient descent convergence. To identify optimal hyperparameters, the Optuna library was used to perform hyperparameter optimization over learning rate (lr) and weight decay. The search space for lr ranged from 10^{-5} to 10^{-3}, and for weight decay from 10^{-6} to 10^{-3}, both on a logarithmic scale. The objective function trained a ResNet-18 model for three epochs per trial, evaluating the F1-score on the validation set. The best parameters (e.g., `lr`=1.565×10^{-5}, `weight_decay`=3.597×10^{-6}) were saved to a **JSON** file and used for subsequent training, ensuring reproducibility and optimal performance.

```
import optuna

def objective(trial):
  # Hyperparameter suggestions
  lr = trial.suggest_loguniform('lr', 1e-5, 1e-3)
  weight_decay = trial.suggest_loguniform('weight_decay', 1e-6, 1e-3)

  # Model
  model = models.resnet18(pretrained=True)
  model.fc = nn.Linear(model.fc.in_features, 2)
  model.to(device)

  criterion = SoftF1Loss()
  optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)

  # Train 1 - 3 epochs quickly for evaluation
  train_model(model, train_loader, val_loader, criterion, optimizer, epochs=3)

  # After training, evaluate F1 on val set
  f1 = evaluate_model_on_split('val')["F1-Score"]
  return f1
```

3.6.b.4 Learning Rate Scheduling

A **ReduceLROnPlateau** scheduler was integrated to dynamically adjust the learning rate based on validation loss. The scheduler reduces the learning rate by a factor of 0.5 if the validation loss does not improve for two consecutive epochs (`patience=2`). This adaptive strategy prevents overfitting and helps the model escape local minima, enhancing convergence. The scheduler's `mode='min'` ensures it responds to minimization of the validation loss, aligning with the training objective.

```
from torch.optim.lr_scheduler import ReduceLROnPlateau
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2)
```

3.6.b.5 Training procedure on Early Stopping

The training loop was designed to monitor both training and validation performance, reporting loss, accuracy, and F1-score per epoch. An early stopping mechanism was implemented to halt training if the validation F1-score did not improve for a specified number of epochs (`patience=4 or 10` in different experiments). The model state with the highest validation F1-score was retained, preventing overfitting and optimizing generalization. The training process was conducted for up to 30 epochs, though early stopping often terminated earlier (e.g., epoch 9 with a validation F1-score of 0.7559).

```
def train_model(model, train_loader, val_loader, criterion, optimizer,
scheduler=None, epochs=10, patience=3):
  best_f1 = 0.0
  epochs_no_improve = 0
  best_model_state = copy.deepcopy(model.state_dict())

  for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0

    for inputs, labels in train_loader:
      inputs, labels = inputs.to(device), labels.to(device)

      optimizer.zero_grad()
      outputs = model(inputs)
      loss = criterion(outputs, labels)
      loss.backward()
      optimizer.step()
```

```
      running_loss += loss.item() * inputs.size(0)
      correct += (outputs.argmax(1) == labels).sum().item()

    train_loss = running_loss / len(train_loader.dataset)
    train_acc = correct / len(train_loader.dataset)

    # Evaluate on validation set
    # ...

    # Early stopping logic
    if val_f1 > best_f1:
      best_f1 = val_f1
      epochs_no_improve = 0
      best_model_state = copy.deepcopy(model.state_dict())
    else:
      epochs_no_improve += 1
      if epochs_no_improve >= patience:
      print(f"Early stopping triggered at epoch {epoch+1}.
      Best Val F1: {best_f1:.4f}")
    model.load_state_dict(best_model_state)
    break

  return model # Return the best model
```

3.6.c Evaluation and Performance Metrics

Model performance was evaluated on training, validation, and test sets using a comprehensive set of metrics: accuracy, precision, recall, F1-score, AUC-ROC, and confusion matrix. The evaluation function processes each image individually, applying the same pre-processing transformations and computing predictions and probabilities. The final model achieved a validation F1-score of 0.7559, with a test F1-score of 0.7447, indicating robust generalization. The high recall (0.7692 on test) suggests effective detection of cracked surfaces, critical for structural safety applications.

```
from sklearn.metrics import (
  accuracy_score, precision_score, recall_score,
  f1_score, roc_auc_score, confusion_matrix
)

# Evaluation function
def evaluate_model_on_split(split_name):
```

```
    # Load images and labels
    # Run inference
    # Compute metrics

    return {
      "Accuracy": accuracy_score(y_true, y_pred),
      "Precision": precision_score(y_true, y_pred, zero_division=0),
      "Recall": recall_score(y_true, y_pred, zero_division=0),
      "F1-Score": f1_score(y_true, y_pred, zero_division=0),
      "AUC-ROC": roc_auc_score(y_true, y_score),
      "Confusion Matrix": confusion_matrix(y_true, y_pred).tolist()
    }
```

3.6.c.1 Implementation Considerations

The implementation leverages GPU acceleration when available, using `torch.device("cuda" if torch.cuda.is_available() else "cpu")`. The model state is saved post-training (**`resnet_best_params_early4.pth`**), enabling deployment or further fine-tuning. The use of libraries like Pandas for CSV handling, PIL for image loading, and scikit-learn for metrics ensures a modular and maintainable codebase. The integration of tqdm for progress bars and JSON for hyperparameter storage enhances usability and reproducibility.

3.6.c.2 Discussion and Further Directions

The methodology demonstrates a robust framework for implementing ResNet-18 in crack detection, with experimental modifications improving performance over a baseline approach. The Soft F1 Loss effectively addresses class imbalance, while hyperparameter tuning and scheduling optimize training dynamics. However, the consistent validation F1-score of approximately 0.2645 across Optuna trials suggests potential issues with the optimization setup or dataset characteristics, warranting further investigation. Future work could explore data augmentation to enhance robustness, alternative architectures (e.g., ResNet-50), or ensemble methods to boost performance. Additionally, extending the hyperparameter search to include batch size or scheduler parameters could yield further improvements.

3.6.d MobileNetV3

```
# Get MobileNetV3 model
model = get_MobileNetV3_model(num_classes=2)
# 2 classes: Non-cracked (0) / Cracked (1)
model.to(device)
```

```
# Define loss function
criterion = nn.CrossEntropyLoss()
# Define optimizer and learning rate scheduler
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=0.0001)
# Use learning rate scheduler to reduce lr by 0.1 every 3 epochs
scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.1)
print(f"-----------------------------------------------
----------------------------")
# Train model
print("Starting training...")
start_time = time.time()
model, best_accuracy = train_model(
  model,
  train_loader,
  val_loader,
  criterion,
  optimizer,
  scheduler,
  device,
  num_epochs=10)
end_time = time.time()
print(f"Training completed in {(end_time - start_time) / 60:.2f} minutes")
print(f"Best validation accuracy: {best_accuracy:.2f}%")
torch.save(model.state_dict(), '6040_MobileNetV3_concrete_crack_detector.pth')
print(f"-----------------------------------------------
----------------------------")
```

The function by calling **`get_MobileNetV3_model`** which loads a pretrained MobileNetV3-Large model using default weights. The final classification layer is then replaced to match the number of output classes, which is set to 2 (representing cracked and non-cracked surfaces).

```
# Load pretrained MobileNetV3 model
def get_MobileNetV3_model(num_classes=2):

  weights = MobileNet_V3_Large_Weights.DEFAULT
  model = mobilenet_v3_large(weights=weights)
  model.classifier[3] = nn.Linear(model.classifier[3].in_features, num_classes)

  return model
```

Once the model is prepared, it is moved to the appropriate computation device (either

GPU or CPU) to ensure efficient training. The loss function is defined as cross-entropy loss, which is suitable for multi-class classification tasks. An Adam optimizer is configured to update the model's weights during training, with a small weight decay to prevent overfitting. A learning rate scheduler is also set up to reduce the learning rate every 3 epochs by a factor of 0.1, helping the model converge more effectively.

Next, the training process begins by calling the **`train_model`** function. This function manages the overall training loop over a specified number of epochs. Within each epoch, the model is first trained on the training dataset using the **`train_one_epoch`** function. In this function, the model is set to training mode, and for each batch in the training loader, it performs a forward pass to make predictions, computes the loss, calculates gradients through back-propagation, clips gradients to avoid explosion, and updates the model's weights. It also tracks and prints batch-level accuracy and loss periodically. After training on all batches, the average loss and accuracy for the epoch are returned.

```
# Function to train the model
def train_model(model, train_loader, val_loader, criterion, optimizer,
 scheduler, device, num_epochs):
  best_accuracy = 0.0
  best_model_wts = None

  for epoch in range(num_epochs):
    print(f"Epoch {epoch+1}/{num_epochs}")
    print("-" * 10)

    # Train for one epoch
    epoch_loss, train_accuracy = train_one_epoch(model, train_loader,
     criterion, optimizer, device)
    print(f"Training Loss: {epoch_loss:.4f},
     Training Accuracy: {train_accuracy:.2f}%")

    # Update learning rate
    scheduler.step()

    # Evaluate on validation set
    val_metrics = evaluate(model, val_loader, device)
    val_accuracy = val_metrics['accuracy'].item()
    val_f1 = val_metrics['f1'].item()

    print(f"Validation Accuracy: {val_accuracy:.2f}%, F1-Score: {val_f1:.4f}")

    # Save best model based on accuracy
```

```
    if val_accuracy > best_accuracy:
      best_accuracy = val_accuracy
      best_model_wts = model.state_dict().copy()
      torch.save(best_model_wts, f'best_model_epoch_{epoch+1}.pth')
      print(f"Saved best model with Accuracy: {best_accuracy:.2f}%")

    print()

  # Load best model weights
  model.load_state_dict(best_model_wts)
  return model, best_accuracy
```

```
def train_one_epoch(model, data_loader, criterion, optimizer, device):
  model.train()

  total_loss = 0
  num_batches = 0
  correct = 0
  total = 0

  for images, targets in tqdm(data_loader, desc="Training"):
    try:
      # Move data to device
      images = images.to(device)
      targets = targets.to(device)

      # Zero the parameter gradients
      optimizer.zero_grad()

      # Forward pass
      outputs = model(images)
      loss = criterion(outputs, targets)

      # Backward pass
      loss.backward()

      # Clip gradients to prevent explosion
      torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

      # Update weights
      optimizer.step()
```

```
        # Update statistics
        total_loss += loss.item()
        num_batches += 1

        # Calculate accuracy
        _, predicted = torch.max(outputs.data, 1)
        total += targets.size(0)
        correct += (predicted == targets).sum().item()

        # Print loss occasionally
        if num_batches % 50 == 0:
        accuracy = 100 * correct / total
        print(f"Batch {num_batches}, Loss: {loss.item():.4f},
         Accuracy: {accuracy:.2f}%")

      except Exception as e:
        print(f"Error in batch: {e}")
        continue

    # Calculate average loss and accuracy
    if num_batches == 0:
      print("WARNING: No valid batches in this epoch!")
      return 0.0, 0.0

    avg_loss = total_loss / num_batches
    accuracy = 100 * correct / total

    print(f"Processed {num_batches} batches")
    print(f"Training Loss: {avg_loss:.4f}, Accuracy: {accuracy:.2f}%")

    return avg_loss, accuracy
```

After training each epoch, the learning rate scheduler is updated, and the model is evaluated on the validation dataset using the evaluate function. This function sets the model to evaluation mode and disables gradient computation for efficiency. It then makes predictions for the entire validation set and collects both predictions and ground truth labels to calculate overall accuracy. A confusion matrix is generated to compute precision, recall, and F1-score, which are printed for detailed performance insights. If the current epoch yields the best validation accuracy so far, the model's weights are saved.

```
def evaluate(model, data_loader, device):
```

```
model.eval()
correct = 0
total = 0
all_preds = []
all_targets = []

with torch.no_grad():
  for images, targets in tqdm(data_loader, desc="Evaluating"):
    images = images.to(device)
    targets = targets.to(device)

    # Get predictions
    outputs = model(images)
    _, predicted = torch.max(outputs.data, 1)

    # Update statistics
    total += targets.size(0)
    correct += (predicted == targets).sum().item()

    # Store predictions and targets for additional metrics
    all_preds.extend(predicted.cpu().numpy())
    all_targets.extend(targets.cpu().numpy())

# Calculate accuracy
accuracy = 100 * correct / total

# Calculate confusion matrix
cm = confusion_matrix(all_targets, all_preds)
precision, recall, f1, _ = precision_recall_fscore_support(all_targets,
 all_preds, average='binary')

print(f"Accuracy: {accuracy:.2f}%")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"Confusion Matrix:\n{cm}")

return {
  "accuracy": torch.tensor(accuracy),
  "precision": torch.tensor(precision),
  "recall": torch.tensor(recall),
  "f1": torch.tensor(f1)
```

```
}
```

Once all epochs are completed, the best-performing model (based on validation accuracy) is restored and saved to a file **`MobileNetV3_concrete_crack_detector.pth`** using `torch.save`. This saved model can later be used for testing or deployment.

3.7 Object Detection Models

3.7.a Faster R-CNN

Since we chose to implement a model capable of both object detection and localization, we encountered an error stating: `"Missing bounding box columns: ['x1', 'y1', 'x2', 'y2'] - Creating default bounding boxes for the entire image."`

This highlighted a critical issue that the `label.csv` file on its own is not sufficient for training a Faster R-CNN model. While label.csv typically contains only image filenames and their corresponding class labels. This format is suitable for image classification tasks, not object detection. On the other hand, Faster R-CNN requires precise spatial information specifically, bounding box coordinates (such as $x_{min}, y_{min}, x_{max}, y_{max}$) in order to accurately learn both the location and category of each object within an image. Without this data, the model cannot effectively learn to detect or localize objects, rendering the training process ineffective.

Therefore, unless the CSV file contains both class labels and bounding box annotations, it cannot be used to train a Faster R-CNN model properly.

3.8 Evaluation

To ensure reliable assessment of model performance, we adopted a structured evaluation strategy centered on generalization to unseen data. The dataset was divided into three subsets: 70% for training, 15% for validation, and 15% for testing.

To mitigate class imbalance during training, we applied random undersampling to the training set by reducing the number of non-cracked images to match the cracked ones, resulting in a balanced 50:50 ratio. In contrast, both the validation and test sets retained their close to original distribution (approximately 40:60 cracked to non-cracked). It can reflect real-world conditions. This strategy ensures the model learns from balanced data while still being evaluated on naturally imbalanced data, reducing bias during training and improving the validity of performance assessment on unseen scenarios.

Given the class imbalance in the test set, we selected the F1-score as our primary evaluation metric. Unlike accuracy, the F1-score provides a more informative measure when dealing with imbalanced classes by considering both precision and recall. This

ensures that the model,Äôs ability to correctly identify cracks is properly evaluated, without being biased by the dominant non-crack class.

All reported results in this study are based solely on the unseen test set, which was held out during both training and validation. This strict separation prevents data leakage and provides a clear indication of how well the models generalize to new, real-world data.

3.8.a Custom CNN

To explore the feasibility of a lightweight architecture, we developed a Custom CNN and evaluated it on both 50:50 balanced and 40:60 imbalanced dataset configurations. However, results indicated that this model consistently underperformed across all metrics compared to deeper architectures like ResNet and MobileNetV3.

In the 50:50 setup, the model achieved only 0.74 accuracy, 0.32 precision, 0.63 recall, and an F1-score of 0.43, with an AUC-ROC of 0.63 - demonstrating poor precision and class separation. The performance further declined in the 40:60 configuration, where although recall was high (0.88), the overall accuracy dropped to 0.50, and precision plummeted to 0.21, yielding an F1-score of just 0.34. These results reflect a high false-positive rate and weak generalization ability.

Given its limited effectiveness and significantly weaker performance relative to other models in this study, we decided to discontinue further experiments with the Custom CNN and excluded it from deployment considerations.

Dataset	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Confusion Matrix
50:50	0.74	0.32	0.63	0.43	0.63	5496 1654 469 796
40:60	0.50	0.21	0.88	0.34	0.76	3060 4090 156 1109

Table 3.2: Evaluation result from Custom CNN in Concrete Models

3.8.b ResNet

To assess how dataset balance and hyperparameter tuning affect model performance, we trained and evaluated ResNet on both a balanced 50:50 and imbalanced 40:60 cracked-to-non-cracked dataset configuration. Additionally, we fine-tuned both models using hyperparameter optimization to explore performance ceilings under each setting.

The baseline 50:50 model trained with a fixed learning rate of 0.001 achieved an accuracy of 0.87, a precision of 0.54, and a recall of 0.76, yielding an F1-score of 0.63 and AUC-ROC of 0.89. These results indicate that while the model effectively detected most cracked images (high recall), it also produced a considerable number of false positives,

likely due to limited exposure to non-cracked surface variability during training. However, most evaluation metrics were stronger compared to the 40:60 setup, particularly in precision and F1-score.

After applying hyperparameter optimization, the 50:50 model,Äôs performance significantly improved. It achieved an accuracy of 0.92, precision of 0.72, recall of 0.77, F1-score of 0.74, and AUC-ROC of 0.93. These results demonstrate that with proper tuning, even a model trained on a balanced dataset can generalize well while maintaining high sensitivity and specificity. In comparison, the 40:60 model trained with the same learning rate of 0.001 produced a higher recall of 0.83 and AUC-ROC of 0.91, indicating improved sensitivity and class discrimination. However, its precision dropped to 0.47, and F1-score to 0.60, suggesting a less balanced performance and a higher rate of false positives. This is likely due to the model compensating for the class imbalance during training.

With optimized parameters, the 40:60 model achieved accuracy of 0.90, precision of 0.63, recall of 0.84, F1-score of 0.72, and AUC-ROC of 0.94. This model performed comparably to the optimized 50:50 model, showing superior recall and AUC, which indicates stronger generalization in identifying cracks but at the cost of increased false positives.

Dataset	**LR**	**Acc.**	**Precision**	**Recall**	**F1**	**AUC-ROC**	**Confusion Matrix**
50:50	0.001	0.87	0.54	0.76	0.63	0.89	[6343 807 303 962]
	Best Params	0.92	0.72	0.77	0.74	0.93	[6775 375 292 973]
40:60	0.001	0.83	0.47	0.83	0.60	0.91	[5940 1210 210 1055]
	Best Params	0.90	0.63	0.84	0.72	0.94	[6525 625 197 1068]

Table 3.3: Evaluation result from ResNet in Concrete Models

In conclusion, the 50:50 training configuration with optimized parameters produced the best overall balance between sensitivity and precision, achieving strong F1-score and accuracy. While the 40:60 model offered higher recall and AUC, it came at the cost of reduced precision, leading to more false positives. This tradeoff is important in real-world scenarios: if minimizing false alarms is critical, as in large-scale infrastructure inspections, the 50:50 setup is more appropriate. On the other hand, if maximizing crack detection is prioritized, the 40:60 model with tuned parameters may be preferred. These findings highlight the importance of balancing dataset composition and model tuning based on deployment goals.

3.8.c MobileNetv3

In order to understand how training set composition affects performance, we compared two models of MobileNetV3 which the first one trained on a 50:50 balanced dataset and another on a less aggressively undersampled 40:60 cracked-to-non-cracked dataset. The evaluation results for the 50:50 model showed a high recall of 0.8744, meaning the model was very sensitive in detecting cracks and correctly identified most cracked images. However, its precision was relatively low at 0.5657 which indicates that many of the images classified as "cracked" were actually non-cracked. Therefore, this leads to a high number of false positives.

Moreover, The F1-score from TEST which balances precision and recall is equal to 0.6869. While the F1-score from TRAIN is equal to 0.87 which is quite different. This suggesting moderate overall overfitting while the accuracy was 0.7556 means that the model correctly predicted about 76% of all images. Despite achieving a strong AUC-ROC score of 0.9528 can reflect excellent class discrimination. However, the model exhibited signs of overfitting. This likely resulted from the aggressive undersampling which removed many non-cracked examples and limiting the model,Äôs exposure to natural surface variability. As a result, the model appeared to overly favor the "crack" class which might reduce its reliability in real-world applications where false alarms can be costly and time-consuming.

In contrast, the 40:60 dataset configuration applied less aggressive undersampling, maintaining more non-cracked images and creating a cracked to non-cracked ratio of approximately 2:3. This provided the model with a richer variety of non-cracked surface patterns while still addressing class imbalance. The evaluation results showed a more balanced performance. The Recall score dropped to 0.7829 while precision significantly improved to 0.7222 which indicated that fewer false positives.

Moreover, The F1-score was 0.7513 and it can demonstrate the better harmony between sensitivity and specificity. Additionally, the accuracy increased to 0.8022, and the AUC-ROC was 0.8828 which is slightly lower than the 50:50 model but still showing good class separation. The model trained on this configuration showed less overfitting and generalized better to unseen data. Therefore, In practical deployment especially in post-earthquake inspections, a high false-positive rate can overwhelm response teams with unnecessary checks, making precision as critical as recall. These results suggest the model performed more conservatively, reducing false alarms while maintaining strong crack detection which is crucial for real-world reliability.

In conclusion, while the 50:50 model focused on finding as many cracks as possible. But it still made more mistakes by wrongly labeling non-cracked images as cracked. This happened because it didn't see enough examples of non-cracked images during training. On the other hand, the 40:60 model had a better mix of cracked and non-cracked images which can enhance the performance of the model and it makes more accurate predictions

Dataset	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Confusion Matrix
50:50	0.88	0.57	0.87	0.69	0.95	6301 849 159 1106
40:60	0.91	0.67	0.85	0.75	0.96	6615 535 190 1075

Table 3.4: Evaluation result from MobileNetV3 in Concrete Models

with fewer false alarms. Therefore, we can conclude that the 40:60 configuration is more suitable for deployment in real-life structural inspections, especially in high-stakes post-disaster scenarios. These findings underscore the importance of choosing training strategies that can reflect real-world class distributions and evaluation metrics aligned with deployment goals to ensure dependable performance in mission-critical applications.

3.9 Overfitting Analysis

Both the ResNet and MobileNetV3 models show moderate overfitting, with high training performance (ResNet F1-score: 0.9382, MobileNetV3 F1-score: 0.8653) but a noticeable drop in the validation and test sets (ResNet F1-score: 0.7447, MobileNetV3 F1-score: 0.7478). Despite this, the models maintain consistent AUC-ROC scores (0.93) between the validation and test sets, indicating strong generalization. This overfitting can be justified, given that the models will be retrained with real-world data through user input. This iterative retraining is expected to improve generalization further, especially in practical deployment scenarios. While overfitting is present, it does not undermine the models' potential for real-world applications, and the overall performance is deemed suitable for the project's objectives.

3.10 Conclusion

Based on the comprehensive evaluation of the classification models, we have identified clear deployment candidates that demonstrate robust performance across both balanced and imbalanced dataset configurations. Among the architectures tested, ResNet and MobileNetV3 consistently outperformed the Custom CNN, particularly in terms of F1-score, precision, and generalization to unseen data. Their superior results were achieved after extensive hyperparameter tuning and strategic dataset balancing, which enhanced the model's ability to capture the complex visual features necessary for reliable crack detection.

For ResNet, the optimized 50:50 training configuration produced a strong balance between sensitivity and precision, achieving an F1-score of 0.74 and an AUC-ROC of

0.93, making it well-suited for applications where both false positives and false negatives must be minimized. Meanwhile, the 40:60 configuration, while offering slightly higher recall (0.84) and AUC-ROC (0.94), did so at the expense of precision, suggesting a higher false alarm rate. Nevertheless, both configurations performed competitively and provide deployment-ready reliability depending on specific use-case priorities.

MobileNetV3, a more lightweight model, demonstrated excellent adaptability, with the 40:60 configuration yielding the highest accuracy (0.91), a well-balanced F1-score (0.75), and reduced overfitting compared to its 50:50 counterpart. This makes it a particularly viable model for deployment in resource-constrained environments such as mobile or embedded systems used in field inspections. Its improved precision and better generalization in the 40:60 setup make it especially valuable in high-stakes scenarios such as post-earthquake assessments, where reducing false positives is as critical as detecting true cracks.

Therefore, we will proceed to deploy both the optimized ResNet and MobileNetV3 models, leveraging their respective strengths. ResNet will serve as a high-capacity baseline model for environments with sufficient computational resources, while MobileNetV3 offers a compact, efficient alternative optimized for real-world deployment. This dual-deployment strategy ensures robustness, scalability, and adaptability across a range of structural inspection contexts, ultimately contributing to more reliable and actionable damage assessment systems.

Model	Acc.	Prec.	Recall	F1	AUC-ROC	**Confusion Matrix**	**Based On**
Custom CNN (50:50)	0.74	0.32	0.63	0.43	0.63	5496 1654 469 796	Lightweight exploration
Custom CNN (40:60)	0.50	0.21	0.88	0.34	0.76	3060 4090 156 1109	Lightweight exploration
ResNet (50:50, LR=0.001)	0.87	0.54	0.76	0.63	0.89	6343 807 303 962	Balanced dataset
ResNet (50:50, Best Params)	0.92	0.72	0.77	0.74	0.93	6775 375 292 973	Tuned for balance
ResNet (40:60, LR=0.001)	0.83	0.47	0.83	0.60	0.91	5940 1210 210 1055	Imbalanced dataset
ResNet (40:60, Best Params)	0.90	0.63	0.84	0.72	0.94	6525 625 197 1068	Tuned for recall
MobileNetV3 (50:50)	0.88	0.57	0.87	0.69	0.95	6301 849 159 1106	Aggressively undersampled
MobileNetV3 (40:60)	0.91	0.67	0.85	0.75	0.96	6615 535 190 1075	Mildly undersampled

Table 3.5: Comparison of Evaluation Result from Crack Detection in Concrete Models

CHAPTER 4

WORKFLOW ORCHESTRATION

To manage the end-to-end machine learning pipeline, Apache Airflow was used as the primary workflow orchestration tool. Our group transitioned from a Jupyter Notebook-based prototype to a modular, production-ready DAG based orchestration system.

4.1 Jupyter to Airflow migration

Refactored original Jupyter code into modular Python scripts (callables), separating concerns such as S3 image loading,data loading and preprocessing, model training, evaluation, Mlflow logging. This ensured each callable is reusable, testable, and easy to fix. We adopted clear naming conventions, parameterization, layers, and logging throughout.

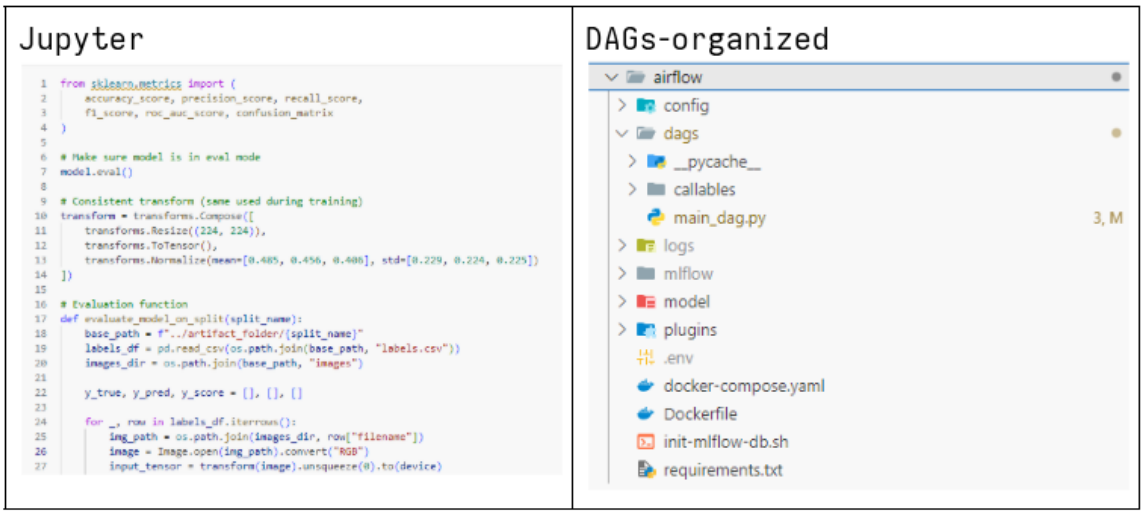

Figure 4.1: Workflow organization using DAG

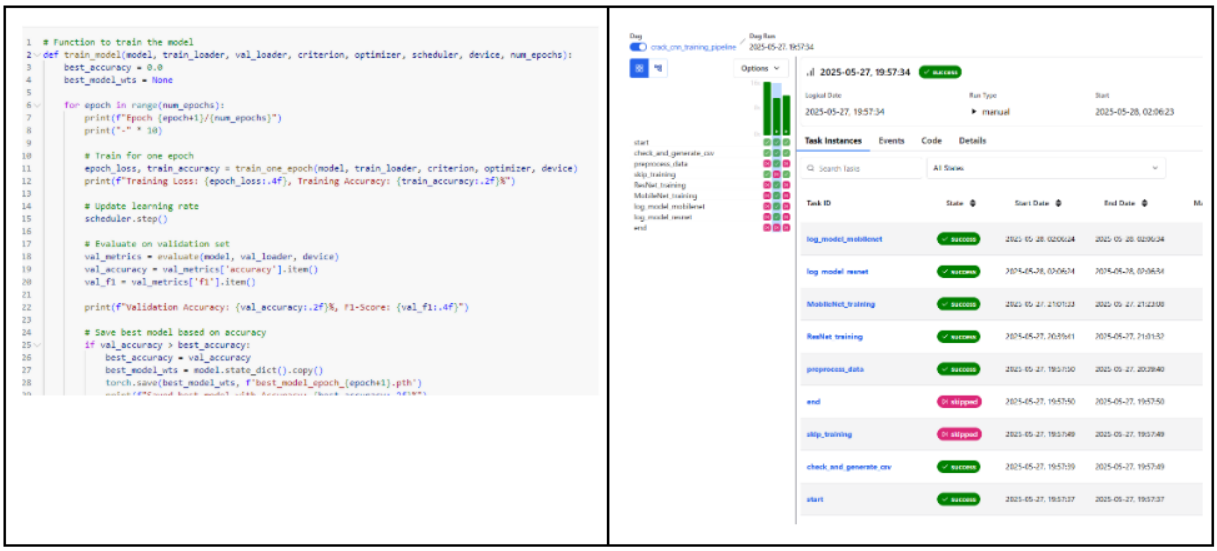

Figure 4.2: Task Scheduling using Airflow

Figure 4.3: Uploaded images from users in Cloudinary S3

4.1.a DAG Structure and Execution

A main DAG (`crack_cnn_training_pipeline`) coordinates tasks like:

- Checking new images (Figure 4.3) from S3 Cloudinary for new user images in the system.
- Data preparation and preprocess. (Figure 4.4)
- Parallel model training (Figure 4.5) or series model training (Figure 4.6) for different resources.
- Evaluation and logging to MLflow (Figure 4.7)

We also utilized Celery and Flower tools. Celery Executor used for scaling out by distributing the workload to multiple Celery workers that can run on different `machinesFlower` is a web based tool for monitoring and administering Celery clusters.

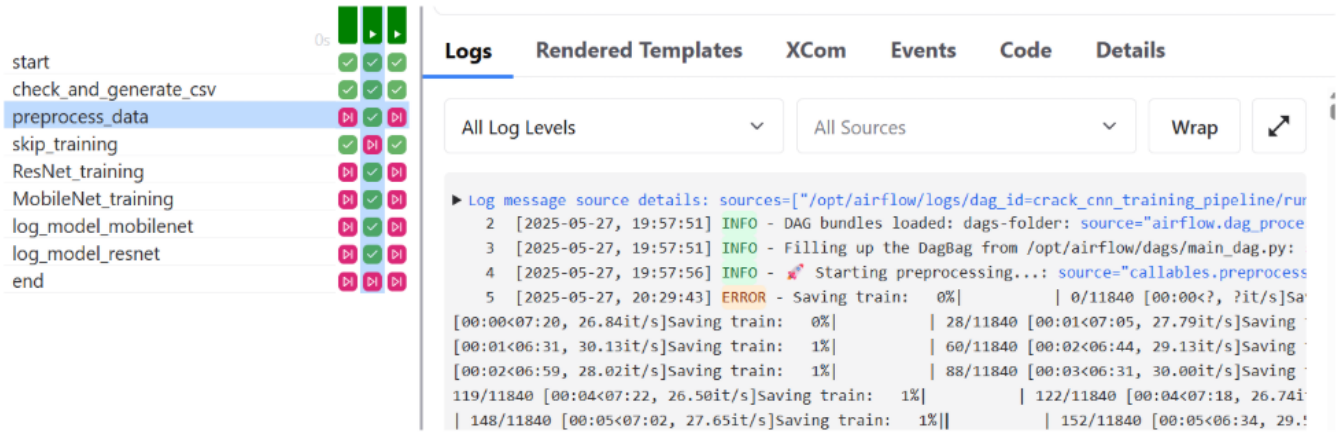

Figure 4.4: Data preparation and preprocess

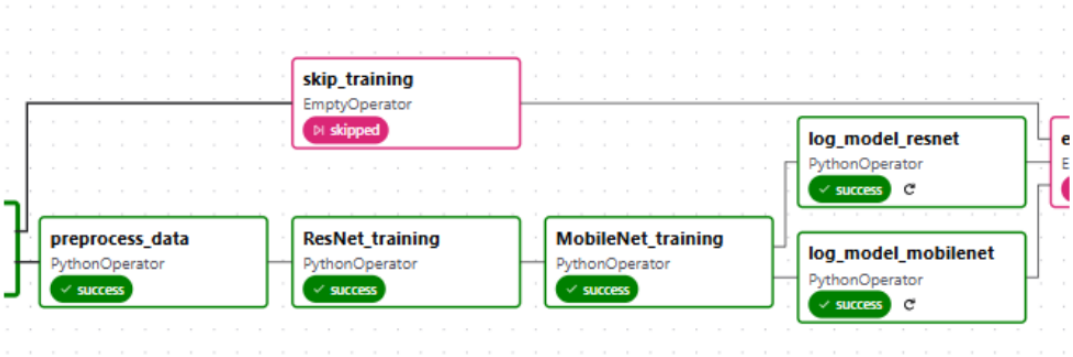

Figure 4.5: Model training pipeline (parallel mode)

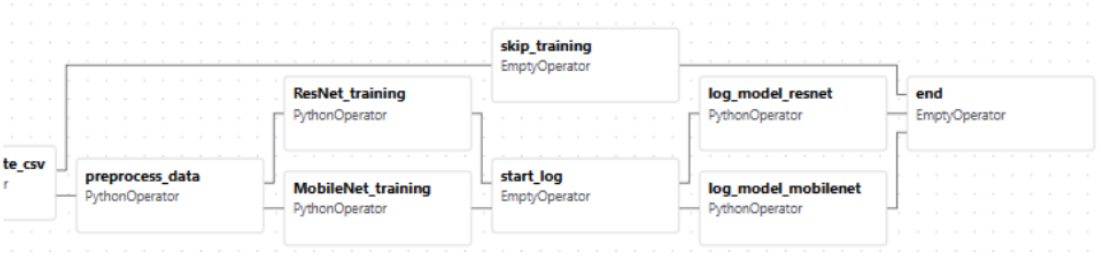

Figure 4.6: Model training pipeline (series mode)

Figure 4.7: Evaluation and logging to MLFlow

Flower can be enable by passing `profile flower` option (e.g., `docker compose up - profile flower up`). This makes the project scalable, parallelized training and distribution. Custom Python Operator tasks manage logic with inter-task communication via `XCom`. For example, we parsed all metrics and values from training operator to logging operator via `XCom`. Moreover, in each operators, we provide a log for monitoring.

4.1.b Fully deployed workflow

- The pipeline is containerized using Docker Compose and runs consistently across environments.
- All Airflow Components (scheduler, webserver, worker, etc.) are all capable for production deployment, running in coordinated services.
- DAGS run automatically on schedule (weekly) for batch continuous learning of the model, with integrated health checks, logs, and retries.
- MLflow logging in integrated as part of the pipeline (as mentioned above in Experiment tracking topic)

4.1.c Clean Code and Organization

For best practices, we modularized all the codes with directory structure, as presented in figure 4.8.

This ensured separation of concerns, readable `docstrings`, and consistent naming. we used `.env` variables and airflow configs for portability, security and scalability.

4.2 Model Experiment Tracking using MLFlow

In this project, MLflow is used for managing machine learning experiment tracking. MLflow provides a centralized interface to log, visualize, and compare different training runs systematically.

Key Features used:

Tracking URI MLflow tracking server was hosted as a Docker container and accessed via the URI `http://mlflow:5000`. This Docker container is network-accessible within the Airflow service. This custom image contains a fully production airflow pipeline and MLflow ready. By running the code below, both Airflow and MLflow will be accessible via `localhost:8000` and `localhost:5000` respectively. To build, simply `cd airflow` and `docker compose up -d –build`.

Logging Parameters Hyperparameters such as optimizer type, learning rate, and number of epochs were logged using `mlflow.log_param()`.

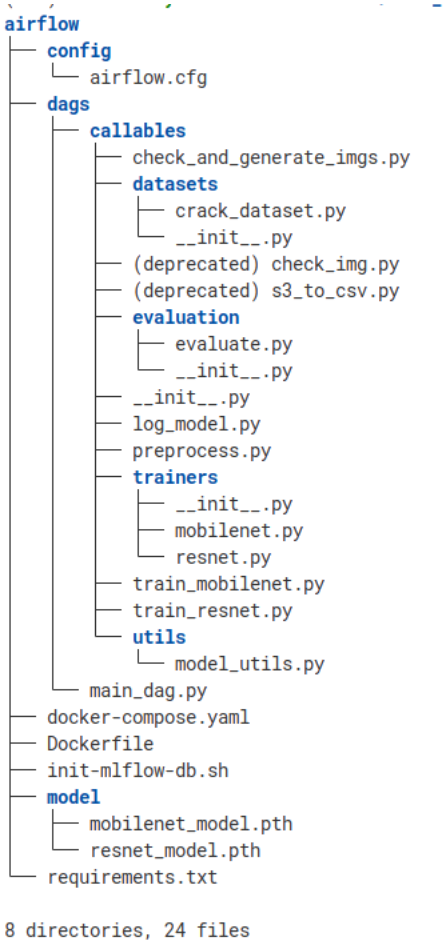

Figure 4.8: Datasets sample overview

Logging Metrics Model metric, such as accuracy, F1, confusion matrix and more, was logged for each experiment using `mlflow.log_metric()`.

Model Artifact Logging Trained models were saved locally using mlflow.pytorch.log_model() with or without registering them in the Model Registry. Artifacts were stored in the shared `/mlflow/artifacts` volume, making it easy to inspect and load models across containers. Although we experienced limitations using the model registry REST endpoint (`/api/2.0/preview/mlflow/registered-models/create`) in this deployment context, the pipeline and MLflow logging are structured to support easy future integration once the endpoint is fully supported by the MLflow server image used.

```
# Example (when enabling model registry)
mlflow.pytorch.log_model(
  model,
  artifact_path="model",
  # registered_model_name="{model_type}-{accuracy}"
  # Uncomment for model registry
```

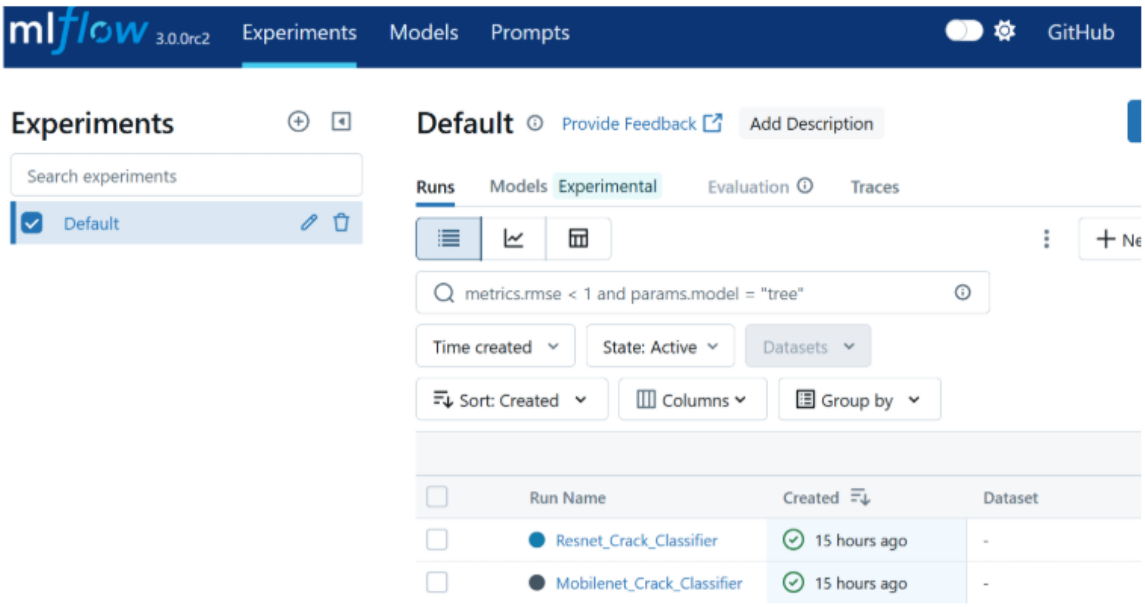

Figure 4.9: Experimental lifecycle

```
)
```

4.3 Experiment Lifecycle

1. Models (i.e., ResNet, MobileNet) were trained with varying configurations and hyparameter-tuned via `optuna`.
2. During each DAG run in Apache Airflow, a `log_model` task handled the MLflow logging for both model.
3. Each run is uniquely identified and can be viewed and compared through MLflow web UI. (`MLFlow image`)
4. Logged models and their metadata are stored for reproducibility and future inference.

Benefits of doing it this way include:

- Centralized tracking of all model training experiments.
- Easy comparison of different architecture and hyperparameter configurations.
- Reproducibility, enabling reruns or deployment of previously trained models.

Figure 4.9 shows experimental life-cycle built on MLFlow.

CHAPTER 5

ML MODEL DEPLOYMENT

To serve the trained deep learning models for inference, we designed a containerized deployment workflow using FastAPI and Docker. This deployment is currently set up to run locally but is structured to be cloud-ready (e.g., AWS, Hugging Face Spaces, or Kubernetes). Therefore, the ecosystem can be scaled horizontally, vertically or deployed with CDN based on traffic, however we can safely anticipate the tool is most likely be used in lab environment by select structural engineering experts. Therefore, local deployment will be sufficient for the time being.

5.1 Deployment Architecture

Figure 5.1 represents the user interface where they can upload the image and provide feedback back to developers through the loop.

`FastAPI` Python based backend service that can load and serve the models.

Docker The FastAPI app is containerized using Docker. The model file, inference code, and model monitoring and workflow orchestration are bundled into another image for reproducibility and ease of deployment.

Shared Volume The model used for inference is saved in a Docker volume (/model) shared between airflow and the deployment container, enabling seam-less hand-off from training to deployment. This is executed in CI/CD pipeline to pull models from airflow shared folder to app folder.

Integration with MLFlow The FastAPI service could be extended to dynamically load models from MLflow artifact store or Model Registry, based on versioning. However, this action is performed using GitHub Action CI/CD during versioned releases.

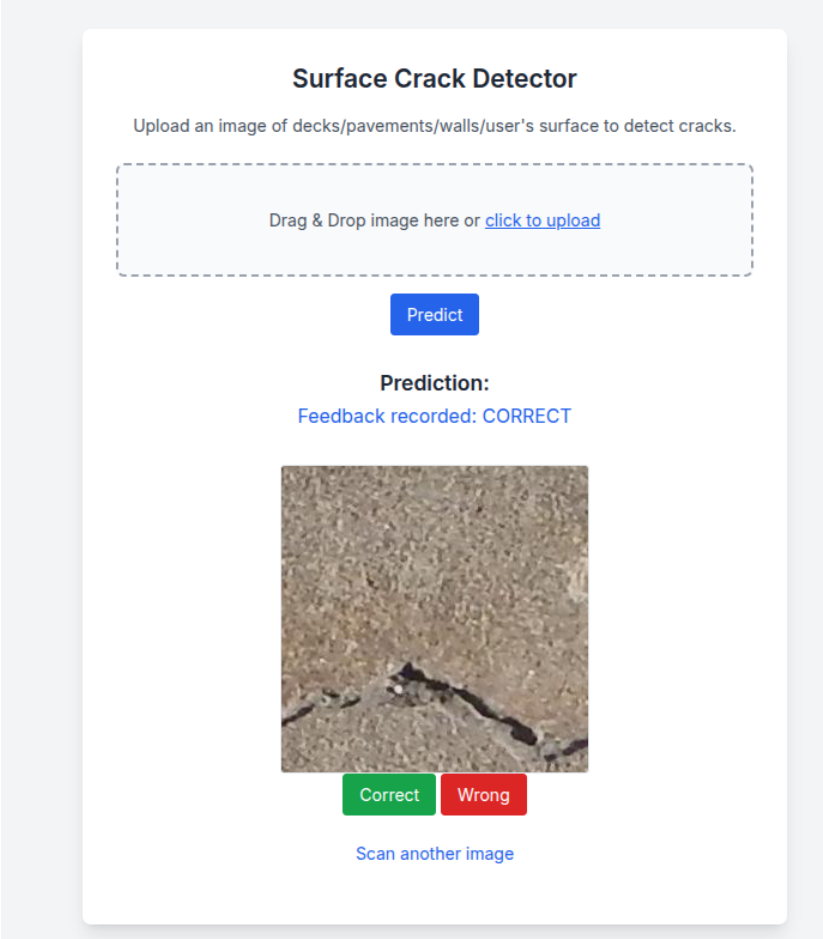

Figure 5.1: Front End for User Application and Feedback Loop

5.2 Containerization

We have individual container setup for the Airflow + MLFlow and the Web app that ships with the freshly released model.

Here's a Dockerfile example we've used in this run. Simply builds a web app that ships with Front End (FastAPI + Jinja).

```
# Base image
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Copy code and the model within
COPY . .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Expose the port
EXPOSE 8000

# Run FastAPI app
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

5.3 Integration with Deployment

Figure 5.2 represents how the entire MLOps project is being carried out. Each block represents a Docker container, while Cloudinary is a 3rd party storage service. User performs inferencing and log is saved on the Cloudinary storage service. Airflow periodically take the newly created artifact the fine tune the model during second run.

This diagram outlines an end-to-end continuous learning workflow for a crack detection system using FastAPI, Cloudinary, and Apache Airflow.

Imagine a user visiting your crack detection web application, where they upload an image of a concrete surface - maybe a photo of a cracked wall, a weathered pavement, or a structure at risk.

As the user submits the image through the web browser, it's received by the server, which quickly runs it through the current best-performing deep learning model - perhaps a ResNet or a MobileNet variant trained for surface crack detection. The server returns a prediction, identifying whether the image shows a cracked or non-cracked surface. But here's where the human-in-the-loop intelligence shines: the user is not just a passive observer. If the prediction is incorrect, they can correct it, helping label the image with the true class.

Once corrected and confirmed, the newly labeled image is stored,Äîuploaded to a Cloudinary S3 bucket that acts as a centralized training repository.

Apache Airflow is tasked with periodically checking the Cloudinary store via an API. It looks for new, user-labeled data. When it detects that a threshold has been met,Äîsay, at least 10 new labeled images,Äîit automatically triggers a retraining pipeline. This pipeline starts with preprocessing the data, ensuring that it,Äôs clean, resized, and ready for ingestion. Then the model retrains, learning from the new examples, refining its internal weights to better understand real-world imperfections.

As training progresses, every experiment is meticulously tracked using MLflow. Accuracy, precision, recall,Äîall logged. If the new model shows an improvement, it proceeds to deployment, replacing the old version on the server.

In this way, the system continuously evolves. Each user contribution not only improves individual predictions but enriches the global model. Thing that started as a simple upload

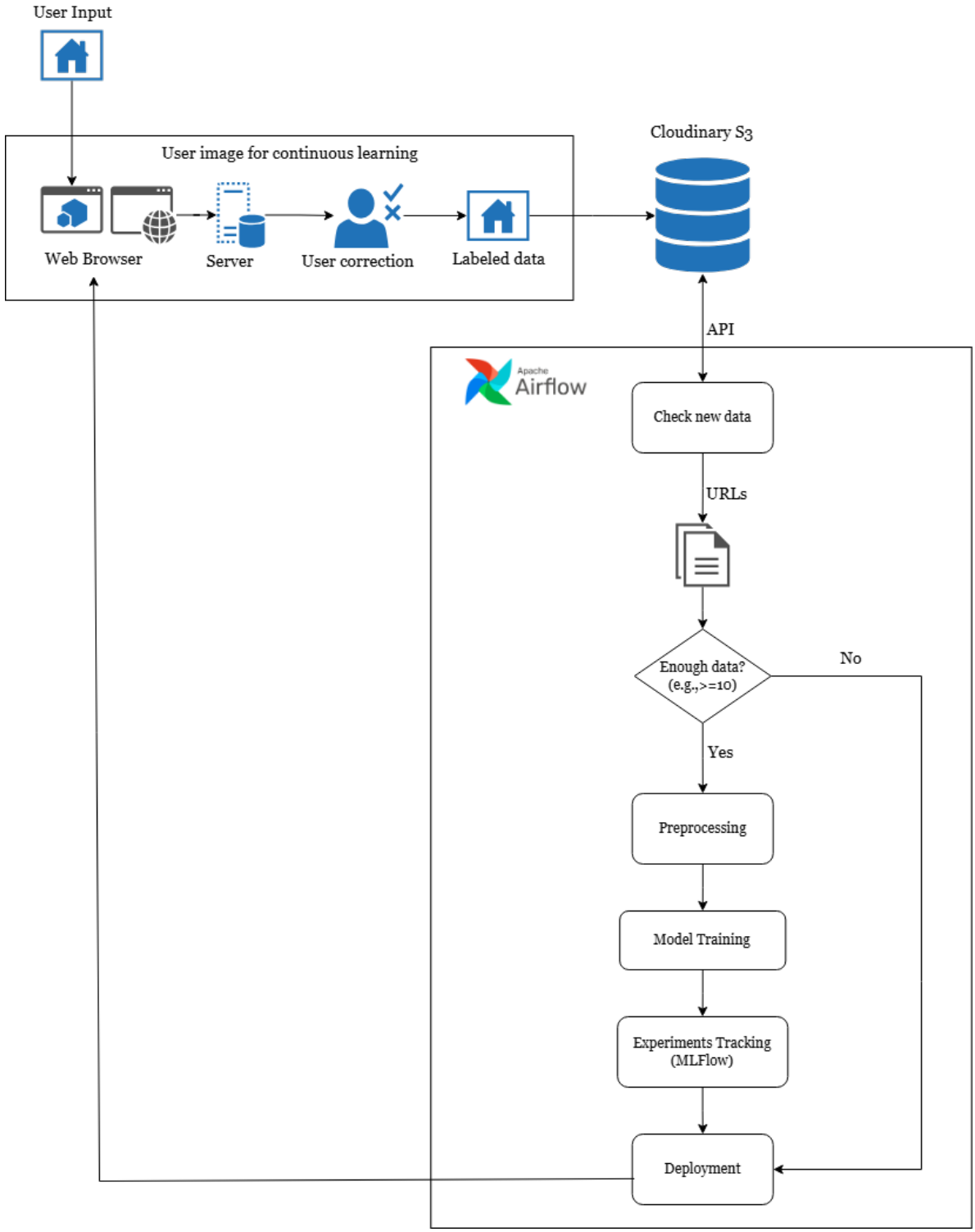

Figure 5.2: Complete MLFlow System

becomes a part of an intelligent, adaptive loop - connecting user insight, cloud storage, and automated learning pipelines, all seamlessly put together.

5.4 GitHub Actions for CI

5.4.a Model selection from MLFlow for new release

When new version needs to be tagged using Git, the CI pipeline will automatically pick the best model prepared using AirFlow+MLFlow side into the app container. The `.pth` serialized files are quite lightweight enought to move across when we release new versions of the models.

```yaml
name: Release on Tag

on:
  push:
    tags:
      - 'v*.*.*' # Triggers on tags like v1.0.0

jobs:
  release-models:
    name: Release Pretrained Models
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Prepare model files
        run: |
          mkdir -p app/models
          cp airflow/model/mobilenet_model.pth app/models/
          cp airflow/model/resnet_model.pth app/models/

      - name: Upload Release Assets
        uses: softprops/action-gh-release@v1
        with:
          name: Release ${{ github.ref_name }}
          tag_name: ${{ github.ref }}
          files: |
            app/models/mobilenet_model.pth
            app/models/resnet_model.pth
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

5.4.b Building front-end

One of the classic use-case of CI, we used it to build Svelte-Kit front-end for our application.

```yaml
name: Build Svelte Frontend (with Bun)

on:
  push:
    paths:
      - "frontend/**"
      - ".github/workflows/frontend-build.yml"

jobs:
  build-frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Install Bun
        run: |
          curl -fsSL https://bun.sh/install | bash
          echo "$HOME/.bun/bin" >> $GITHUB_PATH

      - name: Verify Bun
        run: bun --version

      - name: Install dependencies
        run: bun install

      - name: Build project
        run: bun run build

      - name: Copy to FastAPI static folder
        run: |
          mkdir -p ../app/static
          cp -r build/* ../app/static/
```

5.4.c Unit testing and Linting

To ensure code quality, linting is done on select folder for the Python code to maintain readability.

```yaml
name: Lint Python

on:
  push:
    paths:
      - '**.py'
  pull_request:
    paths:
      - '**.py'

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install ruff
        run: pip install ruff

      - name: Run ruff (lint only one file/folder)
        run: ruff app/main.py # Change to target specific file or directory
```

5.5 Drift detection

Being centered on goals of industrial grade MLOps pipeline, which is the develop ML products and rapidly brind them into production [8], we considered human in the loop process to continuously update the quality of the model. Hence, we do not have enough user feedback to be alarmed to monitor drift in the model since it is very niche area of ML application (civil/structural engineering). Primary kinds of drifts include concept drift, data drift (due to user data), label and feature drift. In the first phase, we

have implemented monitoring of data and model performance. To ensure model remains effective over time, number of corrected labels can be taken into consideration to decide how frequently drift detection should be exercised. As of now, we have Evidently AI as strong choice for model monitoring and drift detection. For the scope of this project, it is omitted and redacted from the documentation considering project goals and development time constraints.

CHAPTER 6

FINDINGS AND DISCUSSIONS

6.1 Findings

The developed system demonstrates an effective integration of image classification, user feedback, and automated machine learning pipelines to enhance surface crack detection in concrete. By involving users in the feedback loop, the system ensures real-world adaptability while minimizing the need for manual relabeling. Performance across various models: ResNet, MobileNet, and Custom CNN shows that:

1. Pretrained architectures like ResNet and MobileNet significantly outperform lightweight custom models, especially under imbalanced data.
2. User corrections effectively enable the accumulation of high-quality labeled data, improving model robustness over time.
3. The deployment of Airflow and MLflow provides a scalable foundation for continuous retraining and experiment tracking.

6.2 Conclusions

This project establishes a full-stack MLOps pipeline capable of:

1. Real-time crack detection via a FastAPI + Jinja web interface.
2. Collecting and incorporating user feedback as labeled training data.
3. Automatically triggering model retraining through Airflow when new data thresholds are met.
4. Tracking model experiments and managing performance history via MLflow.

By closing the feedback loop, the system transforms passive predictions into an active learning pipeline - reducing dataset staleness, improving model accuracy, and enabling data-centric AI development.

6.2.a Limitations

Despite promising results, few limitations were observed:

- The system relies on user honesty for label correction, which may introduce noise.
- Model drift monitoring is not built-in to encounter the first limitation, this shall be the part of future work.
- The system currently supports binary classification (cracked vs non-cracked) and assumes minimal variance in image context.

6.2.b Future works

The system can be extended in a few ways:

- Introduce drift monitoring and eventually replace current log based analytic (feedback loop) to detect drifts or anomalies.
- We haven't successfully tested detection using bounding box in our model (Faster R-CNN) yet, this could allow users to upload images in varied dimensions.

REFERENCES

[1] Ampan Laosunthara, Kodchakorn Krutphong, Natt Leelawat, Wongsa Wararuksajja, Naruethep Sukulthanasorn, Anawat Suppasri, Ratchaneekorn Thongthip, and Chatpan Chintanapakdee. Initial Observations and Immediate Lessons Learned from Thailand's Response to the 2025 Mandalay Earthquake. Online, April 2025. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5229537.

[2] David Forsyth. *Applied Machine Learning*. Springer Nature, Cham, Switzerland, 1 edition, July 2019. doi: 10.1007/978-3-030-18114-7. URL https://link.springer.com/book/10.1007/978-3-030-18114-7.

[3] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2015. doi: doi.org/10.48550/arXiv.1512.03385. URL https://api.semanticscholar.org/CorpusID:206594692.

[4] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 09–15 Jun 2019. URL https://proceedings.mlr.press/v97/tan19a.html.

[5] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3, 2019. URL https://arxiv.org/abs/1905.02244.

[6] Siying Qian, Chenran Ning, and Yuepeng Hu. MobileNetV3 for Image Classification. In *2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, pages 490–497, 2021. doi: 10.1109/ICBAIE52039.2021.9389905. URL https://ieeexplore.ieee.org/document/9389905.

[7] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In C. Cortes and N. Lawrence and D. Lee and M. Sugiyama and R. Garnett, editor, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates,

Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf.

[8] Dominik Kreuzberger, Niklas Kuhl, and Sebastian Hirschl. Machine Learning Operations (MLOps): Overview, Definition, and Architecture, 2022. URL https://arxiv.org/abs/2205.02302.